What an Autonomous Agent Discovers About Molecular Transformer Design: Does It Transfer?

Edward Wijaya
StemRIM, Inc.
wijaya@stemrim.com

Abstract

Deep learning models for drug-like molecules and proteins overwhelmingly reuse transformer architectures designed for natural language, yet whether molecular sequences benefit from different designs has not been systematically tested. We deploy autonomous architecture search via an agent across three sequence types — SMILES, protein, and English text as control — running 3,106 experiments on a single GPU. For SMILES, architecture search is counterproductive: tuning learning rates and schedules alone outperforms the full search ($p = 0.001$). For natural language, architecture changes drive 81% of improvement ($p = 0.008$). Proteins fall between the two. Surprisingly, although the agent discovers distinct architectures per domain ($p = 0.004$), every innovation transfers across all three domains with <1% degradation, indicating that the differences reflect search-path dependence rather than fundamental biological requirements. We release a decision framework and open-source toolkit for molecular modeling teams to choose between autonomous architecture search and simple hyperparameter tuning.

1 Introduction

Transformer architectures developed for natural language processing have been widely adopted for molecular modeling, from property prediction [Chithrananda et al., 2020, Ross et al., 2022] to protein language modeling [Lin et al., 2023] and 3D molecular representations [Zhou et al., 2023]. In nearly all cases, the transformer architecture is borrowed from NLP with minimal modification, implicitly assuming that molecules and language share the same computational requirements. Yet molecular sequences differ from natural language in fundamental ways: SMILES strings use fewer than 50 characters, protein sequences use 20 amino acid tokens, and both are far shorter than typical NLP contexts. Whether these differences warrant distinct architectural designs has not been systematically investigated.

Neural architecture search (NAS) offers a principled way to discover domain-specific designs [Elsken et al., 2019], but traditional NAS operates over hand-designed search spaces that may miss important structural innovations. Recent work has shown that large language models can serve as program search agents, proposing open-ended code modifications that go beyond predefined search spaces [Chen et al., 2023, Romera-Paredes et al., 2024, Hu et al., 2025]. The autoresearch framework [Karpathy, 2026] demonstrated that an LLM agent can iteratively improve a transformer training script for NLP through autonomous code modifications. However, this line of work has been limited to natural language, and no prior work has decomposed how much of the improvement comes from architectural changes versus hyperparameter tuning.

This paper makes three contributions:

1. **A controlled decomposition of architecture search vs. HP tuning.** We design a 4-condition experiment (LLM-guided agent, random NAS, HP-only agent, fixed default) that

Preprint.

cleanly separates the contributions of architecture search and hyperparameter tuning across three domains (SMILES, protein, NLP), totaling 3,106 experiments.

2. **Evidence that the value of architecture search is domain-dependent.** On NLP, architecture search contributes 81% of total improvement over the baseline ($p = 0.008$); on SMILES, HP tuning alone captures 151% of the improvement ($p = 0.001$) and architecture search is counterproductive. For proteins, margins are too small for either component to reach significance.
3. **A surprising universality finding.** While agent-discovered architectures cluster by domain ($p = 0.004$), all 41 innovations transfer across domains with $<1\%$ degradation ($p = 2 \times 10^{-19}$ against the predicted 35% universal rate). The clustering reflects search path dependence, not fundamental domain requirements at this scale.

These findings yield a practical decision framework: for domains with small vocabularies and short sequences (like SMILES), practitioners should constrain their search to hyperparameters only; for domains with large vocabularies and long sequences (like NLP), full architecture search accounts for the majority of gains. We release the complete framework, all 3,106 experiment logs, and the agent prompts as open-source resources.

2 Related Work

LLM-guided program search. The use of LLMs as search agents for code-level optimization has emerged as an alternative to traditional NAS. EvoPrompting [Chen et al., 2023] combines LLM code generation with evolutionary search for neural architecture design, while FunSearch [Romera-Paredes et al., 2024] uses LLMs to discover novel mathematical programs. LM-Searcher [Hu et al., 2025] applies LLM-based NAS across multiple architecture domains. Self-Refine [Madaan et al., 2023] provides a general framework for iterative LLM refinement, and IMPROVE [Xue et al., 2025] extends this to ML code optimization. The AI Scientist [Lu et al., 2024] pursues fully autonomous scientific discovery including architecture design. ELM [Lehman et al., 2022] and LLMatic [Nasir et al., 2023] explore evolution-through-LLMs and quality-diversity optimization respectively. We differ in two respects: (1) we apply LLM-guided search to molecular domains, not just NLP, and (2) we introduce a 4-condition design that decomposes architecture search from HP tuning, a distinction absent from prior work.

Autoresearch. Our framework builds on the autoresearch system [Karpathy, 2026], which demonstrated that an LLM agent can iteratively improve an NLP training script through autonomous code modifications. We extend autoresearch to molecular domains and, critically, add three baseline conditions that enable a controlled decomposition. The original autoresearch work does not separate architectural improvements from hyperparameter tuning, making it impossible to assess the value of architecture search per se.

Neural architecture search. Traditional NAS methods operate over discrete [Elsken et al., 2019] or continuous search spaces with fixed parameterizations. Random search [Bergstra and Bengio, 2012] remains a strong baseline, and methods like BOHB [Falkner et al., 2018] combine Bayesian optimization with bandit-based early stopping. Our random NAS baseline follows this tradition: uniform sampling from a discrete architectural space. The key distinction of LLM-guided search is that it operates on source code directly, enabling open-ended modifications (e.g., introducing gated MLPs or changing residual connections) that lie outside any predefined search space.

Molecular transformers. Transformer-based models for molecular data have proliferated in recent years. ChemBERTa [Chithrananda et al., 2020] and Chemformer [Irwin et al., 2022] apply BERT- and encoder-decoder architectures to SMILES strings. MoLFormer [Ross et al., 2022] introduces linear attention for molecular representations. ESM-2 [Lin et al., 2023] trains protein language models at billion-parameter scale. Uni-Mol [Zhou et al., 2023] incorporates 3D structural information. These models all adopt NLP architectures with minimal domain-specific adaptation. Our work provides the first systematic evidence for *when* such adaptation matters: architecture search adds substantial value for NLP-like domains but not for SMILES-like domains at the $\sim$10M parameter scale.

Scaling laws and compute-optimal training. Scaling laws [Kaplan et al., 2020, Hoffmann et al., 2022] inform the relationship between model size, data, and compute. Our experimental design operates in a deliberately small-scale regime ($\sim$8.6M parameters, 5-minute training) to maximize the

Table 1: Experimental conditions. Each condition controls for a different factor in the search process. Together, the four conditions enable a clean decomposition of architecture search versus hyperparameter tuning contributions.

Condition	Search space	Search strategy	Purpose
Agent	Architecture + HP	LLM-guided	Full capability
Random NAS	Architecture + HP	Uniform random	Controls for search strategy
HP-only	Hyperparameters only	LLM-guided	Controls for architecture search
Fixed default	None	None	Baseline floor

Table 2: Data tracks. Each track uses a different tokenization and sequence length, but shares the same baseline model architecture (Section 3.3). Run counts vary by track to balance statistical power against compute cost.

Track	Dataset	Vocab	Seq len	Agent	NAS	HP-only
SMILES	ZINC-250K [Irwin and Shoichet, 2005]	37	256	5 runs	3 runs	3 runs
Protein	UniRef50 [Suzek et al., 2015]	24	512	3 runs	3 runs	3 runs
NLP	FineWeb-Edu [Penedo et al., 2024]	∼8K (BPE)	2,048	5 runs	3 runs	3 runs

number of architecture evaluations within a fixed budget. This mirrors the proxy-based evaluation common in NAS, where short training runs approximate longer training [Elsken et al., 2019]. We validate our 5-minute proxy against 2-hour training (Spearman $\rho = 0.54$) and discuss its limitations.

Grouped query attention and gated MLPs. Two of the universally beneficial innovations our agent rediscovered, grouped query attention (GQA) [Ainslie et al., 2023, Touvron et al., 2023] and gated linear units [Dauphin et al., 2017, Shazeer, 2020], are well-established in the NLP literature. That the agent converges on these independently, without explicit prompting, supports the finding that effective transformer innovations are largely domain-agnostic at this scale.

3 Methodology

We design a controlled experiment to decompose the contributions of architecture search versus hyperparameter tuning across three sequence domains. The experiment uses four conditions, three data tracks, and a total of 3,106 training runs.

3.1 Experimental Design

Evaluating LLM-guided architecture search requires disentangling its components: does the agent improve performance through architectural modifications, hyperparameter tuning, or the interaction of both? We address this with a 4-condition factorial design (Table 1).

This design enables four pairwise comparisons that isolate specific factors:

- **Agent vs. Random NAS**: value of LLM guidance (holding search space constant)
- **Agent vs. HP-only**: value of architecture search (holding search strategy constant)
- **HP-only vs. Fixed default**: value of hyperparameter tuning alone
- **Random NAS vs. Fixed default**: value of any architecture variation

The key derived quantity is the *decomposition*: for each domain, we partition the total improvement (fixed default → best agent) into an HP contribution (fixed default → best HP-only) and an architecture contribution (best HP-only → best agent). This directly answers whether architecture search adds value beyond hyperparameter tuning.

3.2 Tracks and Data

We evaluate across three sequence domains chosen to span a range of vocabulary sizes and sequence lengths (Table 2).

Table 3: Baseline model architecture. Parameters shared across all three tracks except where noted.

Parameter	Value	Notes
Depth (n_{layer})	6	Reduced from 8 for A10G throughput
Width (n_{embd})	320	Derived from aspect ratio 48, head dim 64
Heads (n_{head})	5	$n_{\text{embd}}/\text{head_dim} = 320/64$
KV heads ($n_{\text{kv_head}}$)	5	Full multi-head attention (not GQA)
FFN multiplier	$5\times$	Hidden dim = $5 \times 320 = 1{,}600$
Activation	ReluSquared	$\text{ReLU}(x)^2$; inherited from autoresearch
Normalization	RMSNorm	Pre-attention and pre-MLP
Attention	SDPA	Flash Attention 2 backend (Ampere)
Window pattern	SSSL (3 short + 1 long)	3 short-window + 1 full-causal per cycle
Positional encoding	RoPE	Rotary position embeddings
Value embeddings	Alternating layers	ResFormer-style input-dependent gating
Weight tying	Disabled	Separate embedding and unembedding
Parameters	$\sim$8.6M	Varies slightly by vocabulary size

SMILES. We use ZINC-250K, a curated subset of 250,000 drug-like molecules represented as SMILES strings [Weininger, 1988]. Tokenization is character-level (37 unique tokens including special characters). We augment with SMILES enumeration (randomized atom orderings) to increase effective training set size. Sequences are padded to 256 tokens.

Protein. We sample 50,000 sequences from UniRef50, a non-redundant protein sequence database clustered at 50% identity. Tokenization is character-level over the 20 standard amino acids plus 4 special tokens. Sequences are padded to 512 tokens.

NLP. We use a subset of FineWeb-Edu, a curated English web text corpus. Tokenization uses byte-pair encoding (BPE) with a vocabulary of approximately 8,192 tokens. Sequence length is 2,048 tokens. This track serves as a control domain where architecture search is known to be effective [Karpathy, 2026].

All tracks use a fixed 90/10 train/validation split. The evaluation metric is validation bits-per-byte (val_bpb), which normalizes cross-entropy loss to a byte-level unit for comparability within each track. We do *not* compare absolute val_bpb values across tracks, as the three domains have different intrinsic entropy.

3.3 Baseline Model Architecture

All four conditions share the same starting architecture: a decoder-only Transformer [Vaswani et al., 2017] autoregressive model adapted from the autoresearch framework [Karpathy, 2026]. The original 50.3M-parameter architecture was designed for H100 GPUs and scaled down to $\sim$8.6M parameters to achieve 3–8 training epochs within the 5-minute budget on A10G hardware (Section 3.4).

Table 3 summarizes the shared architectural parameters. Only data-facing parameters (vocabulary size, sequence length, device batch size) differ across tracks. This ensures that any architectural divergence discovered by the agent is attributable to domain-specific optimization, not baseline differences.

The baseline was intentionally not pre-optimized for molecular data: parameters like the activation function (ReluSquared), FFN ratio ($5\times$), and attention pattern (SSSL) were kept at their autoresearch defaults to give the agent room to discover domain-specific improvements. Starting from a domain-agnostic baseline is essential to the experimental design; a pre-optimized molecular architecture would conflate human domain knowledge with agent discovery.

3.4 Training Procedure

All experiments run on a single NVIDIA A10G GPU (24 GB VRAM) with a fixed 5-minute training budget. The optimizer is MuonAdamW [Jordan et al., 2024]: Muon (orthogonalized momentum with Newton-Schulz iterations) for 2D matrix parameters in transformer blocks, and AdamW for embeddings, scalars, and biases. Default hyperparameters are: embedding LR 0.6, unembedding LR 0.004, matrix LR 0.04, scalar LR 0.5, weight decay 0.2 (linearly decayed), Adam betas $(0.8, 0.95)$,

and warmdown ratio 0.5 with linear cooldown. The total batch size is 65,536 tokens with gradient accumulation (device batch sizes of 256, 128, and 32 for SMILES, protein, and NLP respectively). Training uses `torch.compile` and bfloat16 mixed precision.

At the 5-minute mark, training halts and the model is evaluated on the held-out validation set. The evaluation computes val_bpb over $\sim$655K tokens (5 batches of 131,072 tokens). The A10G achieves $\sim$2.9 epochs over the SMILES training set in this budget, placing the model in a regime where architecture quality differences are discriminable above training noise.

3.5 Search Process

Agent condition. The LLM agent (OpenAI Codex, powered by GPT-5.4) receives a system prompt (`program.md`) specifying the search loop: (1) inspect current architecture and prior results, (2) make one coherent modification to `train.py`, (3) train for 5 minutes, (4) evaluate val_bpb, (5) keep the change if val_bpb improves, otherwise revert. The prompt encourages architectural exploration (depth/width balance, attention patterns, head structure, activation functions, normalization, residual pathways, and embedding structure) while permitting hyperparameter changes. Each run executes $\sim$100 sequential experiments. The agent operates in an isolated workspace and can only modify `train.py`; the data pipeline, evaluation, and training budget are fixed.

HP-only condition. Identical to the agent condition, except the system prompt (`program_hponly.md`) explicitly forbids architectural modifications: "Do NOT change model architecture: no new layers, no attention pattern changes, no activation function changes, no model structure changes." The agent may only modify hyperparameters: learning rates, batch size, weight decay, warmup schedule, and optimizer parameters. This isolates the contribution of architecture search by holding the architecture constant while allowing the same LLM-guided HP optimization.

Random NAS condition. For each of the 100 experiments per run, a random architecture configuration is sampled uniformly: depth $\in [3, 8]$, width $\in \{128, 160, \ldots, 512\}$ (step 32), heads $\in [2, 8]$ (subject to divisibility), activation $\in \{\text{ReLU}, \text{GELU}, \text{SiLU}, \text{ReluSquared}\}$, attention $\in \{\text{full}, \text{windowed}\}$. Each configuration is rendered into `train.py` and trained for 5 minutes. Hyperparameters are held at their default values. This controls for both the LLM search strategy and for the choice of architecture search directions, isolating the value of any non-default architecture.

Fixed default condition. A single training run with the unmodified baseline architecture and default hyperparameters. This provides the floor against which all other conditions are compared. For AUC-OC computation (Section 3.6), the fixed default val_bpb is extended as a flat line over 100 experiment positions.

3.6 Evaluation Metrics

Primary metric: val_bpb. Validation bits-per-byte, computed as cross-entropy loss normalized to byte-level units. Lower is better. Reported per run as the best val_bpb achieved across all experiments in that run.

AUC-OC (Area Under the Optimization Curve). To capture cumulative search efficiency rather than only final performance, we compute the area under the best-so-far curve for each run. At experiment k, the best-so-far value is $\min_{i \leq k} \text{val_bpb}_i$ (excluding crash experiments). The AUC-OC is the trapezoidal integral over experiments 1–100. Lower AUC-OC indicates faster and more efficient search.

Keep rate. The fraction of non-crash experiments where the modification improved val_bpb. This measures the efficiency of the search strategy at proposing beneficial changes.

Decomposition. For each track, the total improvement from fixed default to best agent is decomposed as:

$$\text{total_improvement} = \text{bpb}_{\text{fixed}} - \text{bpb}_{\text{agent}} \tag{1}$$

$$\text{HP_contribution} = \text{bpb}_{\text{fixed}} - \text{bpb}_{\text{hp_only}} \tag{2}$$

$$\text{arch_contribution} = \text{bpb}_{\text{hp_only}} - \text{bpb}_{\text{agent}} \tag{3}$$

where each bpb value is the mean best val_bpb across runs within that condition. By construction, HP_contribution + arch_contribution = total_improvement. The HP contribution percentage can

exceed 100% (and the architecture contribution can be negative) when HP-only search outperforms the full agent.

3.7 Proxy Validation

The 5-minute training budget serves as a proxy for longer training. To validate this proxy, we conducted a calibration study on the SMILES track: 20 diverse architectures (sampled from the same random NAS space) were each trained for both 5 minutes and 2 hours. The rank correlation between 5-minute and 2-hour val_bpb was Spearman $\rho = 0.54$ ($p = 0.014$, $n = 20$), indicating moderate reliability. The proxy preserves coarse architecture rankings but can misorder architectures with similar performance. The proxy is suitable for identifying large improvements but not for fine-grained ranking.

3.8 Statistical Testing

The unit of analysis for between-condition comparisons is the *run* (not the individual experiment), yielding sample sizes of $n = 3$–5 per condition. We employ the following tests:

Bootstrap confidence intervals. For each pairwise AUC-OC comparison, we compute 95% bootstrap CIs over 10,000 resamples. A CI that excludes zero indicates a significant difference. We also report bootstrap p-values (two-sided).

Frequentist tests. Welch's t-test and Mann–Whitney U for AUC-OC and final best val_bpb comparisons. Fisher's exact test for keep rate comparisons. Cohen's d for effect size.

Architecture clustering (H1). We extract architectural feature vectors from the best architecture per agent run (13 total: 5 SMILES + 3 protein + 5 NLP) by parsing the trained `train.py` source code. Features include depth, width, head count, FFN ratio, activation function, attention type, normalization, and optimizer settings. We compute pairwise Gower distances (mixed categorical and numerical features) and test for domain clustering via permutation test: track labels are permuted 10,000 times, and the test statistic is the ratio of mean cross-track to within-track distance.

Domain knowledge rediscovery (H2). We classify each agent modification (from code diffs) against 5 known molecular modeling techniques: local/sliding attention, embedding dimension reduction, positional encoding changes, depth/width rebalancing for short sequences, and regularization for small data. A technique is "matched" in a run if at least one kept experiment implements it.

Multiple comparison correction. All frequentist p-values are corrected using the Holm–Bonferroni method within logical families: H1 (1 test), H2 (5 tests), H4 per-track primary comparisons (5 tests $\times$ 3 tracks), and H4 decomposition (9 tests). Both raw and adjusted p-values are reported. Table 9 in Appendix A.1 provides a complete summary of all hypotheses, predictions, and outcomes.

4 Results

We begin with the decomposition (the central finding), followed by search efficiency, architecture clustering, transfer universality, domain knowledge rediscovery, and downstream validation.

4.1 Decomposition: Architecture Search vs. HP Tuning

Table 4 and Figure 1 show the decomposition of total improvement into HP and architecture contributions for each track.

The three tracks yield qualitatively different decompositions:

NLP (large vocabulary, long sequences). Architecture search contributes 81% of the total 0.030 bpb improvement ($p_{\text{adj}} = 0.009$), with HP tuning contributing only 19% ($p_{\text{adj}} = 0.022$). The full agent achieves a mean best val_bpb of 1.123, compared to 1.147 for HP-only and 1.153 for the fixed default. Architecture search drives most of the gain in this domain.

SMILES (small vocabulary, short sequences). HP tuning alone captures 151% of the total 0.010 bpb improvement ($p_{\text{adj}} = 0.001$), meaning the HP-only agent (mean best 0.581) *outperforms* the

Table 4: Decomposition of improvement from fixed default to best agent. HP contribution = improvement from HP tuning alone (Eq. 2); architecture contribution = additional improvement from architecture search beyond HP tuning (Eq. 3). Percentages sum to 100% by construction. Adjusted p-values use Holm–Bonferroni correction within the decomposition family.

Track	**Total impr.**	**HP %**	**HP p_{adj}**	**Arch %**	**Arch p_{adj}**
SMILES	0.0102 bpb	151%	0.001	−51%	0.246
Protein	0.0098 bpb	6%	0.632	94%	0.632
NLP	0.0299 bpb	19%	0.022	81%	0.009

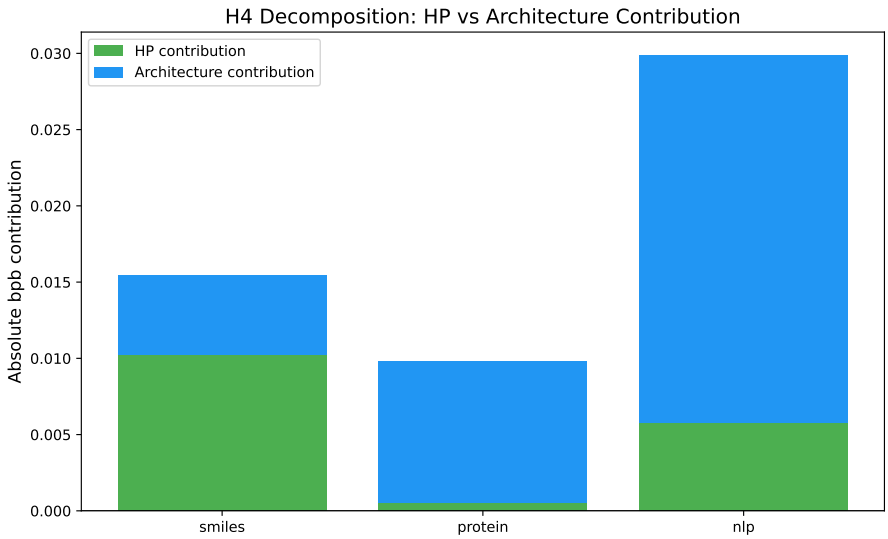


Figure 1: Decomposition of total improvement per track. On NLP, architecture search contributes the majority (81%) of improvement. On SMILES, HP tuning alone exceeds the total improvement (151%), and architecture search is counterproductive (−51%). Protein margins are too small for significance.

full agent (mean best 0.586). The architecture contribution is −51% (not significant, $p_{\text{adj}} = 0.246$), indicating that architecture search wastes experimental budget on structural changes that do not improve over the near-optimal default architecture. This is not a failure of the agent; it is a genuine finding that simple domains do not benefit from architecture search.

Protein (intermediate). The total improvement is 0.010 bpb, but neither the HP contribution (6%, $p_{\text{adj}} = 0.632$) nor the architecture contribution (94%, $p_{\text{adj}} = 0.632$) reaches significance. The relative spread across all conditions is <0.3%, suggesting the domain is "architecture-insensitive" at this model scale.

4.2 Search Efficiency

Table 5 reports pairwise AUC-OC comparisons between the agent and each baseline.

On SMILES, the agent significantly outperforms random NAS ($p_{\text{adj}} = 0.044$, $d = -1.50$), but HP-only significantly outperforms the agent ($p_{\text{adj}} = 0.015$, $d = +1.41$). On NLP, the agent significantly outperforms HP-only ($p_{\text{adj}} = 0.005$, $d = -4.45$), the largest effect size in the study. Agent vs. NAS does not reach significance on NLP after correction ($p_{\text{adj}} = 0.632$), likely because random NAS occasionally samples effective architectures. On protein, no comparison reaches significance.

Figure 2 shows best-so-far trajectories. On SMILES, the HP-only agent converges fastest and to the lowest val_bpb. On NLP, the full agent's curve separates from all baselines by experiment ∼20 and continues to improve. On protein, all conditions converge to a narrow band.

Table 5: Agent vs. baselines: AUC-OC comparisons. Bootstrap 95% CIs from 10,000 resamples; Cohen's d for effect size. Adjusted p-values use Holm–Bonferroni correction within each track's primary comparison family. Negative AUC difference = agent is better (lower AUC).

Comparison	Track	AUC 95% CI	p_{adj}	Cohen's d	Sig.?
Agent vs. NAS	SMILES	$[-0.66, -0.16]$	0.044	-1.50	Yes
	Protein	$[-0.42, -0.17]$	0.137	-2.86	No
	NLP	$[-0.54, +0.31]$	0.632	-1.35	No
Agent vs. HP-only	SMILES	$[+0.55, +1.03]$	0.015	$+1.41$	Yes*
	Protein	$[-0.77, +0.01]$	0.635	-1.07	No
	NLP	$[-2.05, -1.23]$	0.005	-4.45	Yes

*HP-only outperforms agent on SMILES.

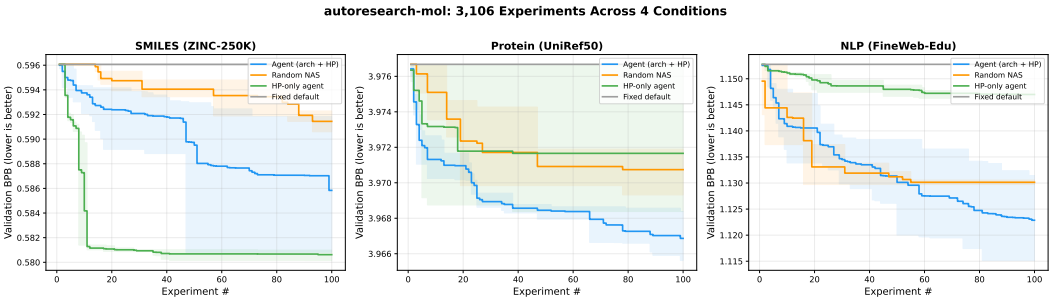


Figure 2: Best-so-far curves across conditions and tracks. Each line shows the mean cumulative minimum val_bpb over experiments 1–100, with shaded bands indicating the min–max range across runs. The HP-only agent (green) dominates on SMILES, the full agent (blue) dominates on NLP, and all conditions cluster tightly on protein.

4.3 Architecture Clustering

We extract architectural feature vectors from the best-performing architecture in each of the 13 agent runs (5 SMILES + 3 protein + 5 NLP). A permutation test on the Gower distance matrix yields $p = 0.004$ (10,000 permutations), with an observed cross-track/within-track distance ratio of 1.43. Agent-discovered architectures cluster by domain (Figure 3).

The qualitative patterns differ across domains. SMILES architectures tend toward shallower, wider configurations with full attention and gated MLPs (SwiGLU). NLP architectures favor aggressive KV head compression (GQA with $n_{\text{kv_head}} = 1$), maintaining full attention across layers. Protein architectures show more variation, with per-layer value head modifications and alternating window patterns.

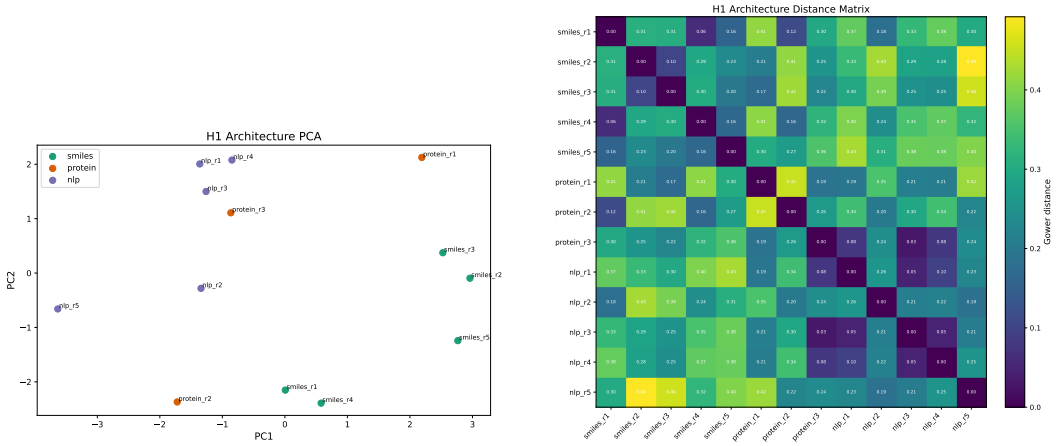
4.4 Transfer Universality

Cross-domain transfer experiments show that architectural innovations are largely universal at this scale, despite the clustering result above. Table 6 and Figure 4 show the 3×3 transfer matrix: training a model with one domain's best architecture on another domain's data yields <1% degradation in most cases.

We tested four sub-hypotheses about transfer:

H3a: Asymmetric transfer. Not supported. All degradation values are below 1%, and some transfers improve performance (e.g., protein architecture on SMILES data: -0.71%).

H3b: Layer specificity. Partially supported. Layer freezing experiments show monotonic degradation as more layers are frozen from deep to shallow, with early layers transferring cleanly (<5% degradation) and late layers showing mild domain specificity (up to 16% for NLP→SMILES at full freeze).



(a) PCA of architectural features

(b) Gower distance matrix

Figure 3: Architecture clustering by domain ($p = 0.004$). (a) PCA projection of 13 best architectures, colored by track. (b) Pairwise Gower distance matrix ordered by track, showing within-track similarity (darker blocks on diagonal).

Table 6: Cross-domain transfer matrix. Each cell shows the relative change in val_bpb when using one domain's best architecture on another domain's data, compared to the native architecture. Negative values indicate improvement.

Architecture → Data	**SMILES**	**Protein**	**NLP**
SMILES arch	—	−0.08%	−0.02%
Protein arch	−0.71%	—	+0.80%
NLP arch	+0.05%	−0.15%	—

H3c: Length matching. Not supported. Truncating NLP sequences to shorter lengths to match molecular domains actively hurts transfer performance (mean reduction −151%), suggesting context window size is a genuine architectural constraint rather than an artifact.

H3d: Innovation classification. Strongly contradicts predictions. All 41 of 41 discovered innovations are classified as universal (<1% degradation when transferred), yielding $p = 2 \times 10^{-19}$ against the predicted 35% universal rate via binomial test.

Clustering (Section 4.3) combined with universal transfer creates an apparent paradox: the agent discovers different architectures for different domains, yet the innovations themselves are domain-agnostic. We interpret this as evidence that search path dependence, what the agent tries first conditioned on the data it observes, drives the apparent specialization rather than fundamental domain requirements at this ∼10M parameter scale.

4.5 Domain Knowledge Rediscovery

We classified agent modifications against 5 known molecular modeling techniques by parsing code diffs from all SMILES agent runs. Of the 5 techniques, 4 were partially matched across all 5 runs: local/sliding attention patterns, embedding dimension adjustments, depth/width rebalancing for short sequences, and gated MLP introduction. Only positional encoding changes were rarely observed (1/5 runs). The agent independently converged on established domain-relevant patterns (gated MLPs, attention pattern modifications, depth/width rebalancing) without domain-specific prompting.

These same techniques also appeared in NLP runs (all 5 runs matched 3/5 techniques), consistent with the universality finding: the innovations the agent discovers for molecular data are not specific to molecules.

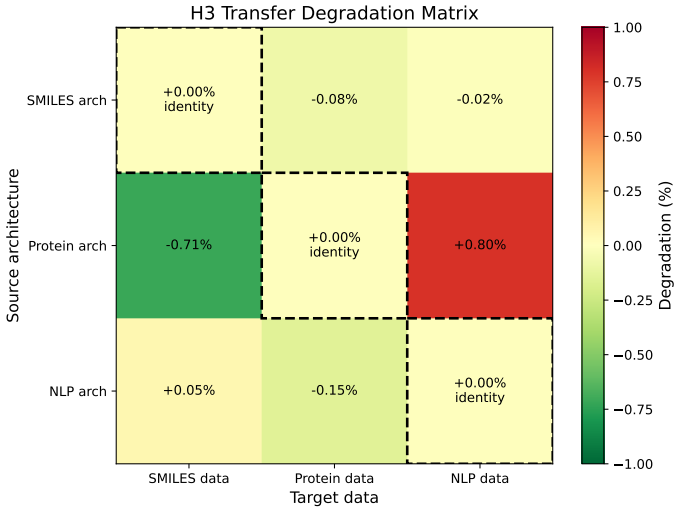


Figure 4: Transfer heatmap. Degradation is below 1% for all cross-domain pairs, with several transfers actually improving performance (negative degradation).

Table 7: MoleculeNet downstream validation. ROC-AUC on three binary classification tasks using pretrained SMILES agent architectures with linear probing.

Architecture	**BBBP**	**HIV**	**BACE**	**Mean**
Agent #1 (bpb 0.581)	0.702	0.758	0.805	0.755
Agent #2 (bpb 0.583)	0.690	0.731	0.795	0.739
Agent #3 (bpb 0.584)	0.711	0.735	0.798	0.748

4.6 Downstream Validation

To verify that val_bpb improvements translate to practical utility, we evaluated the three best SMILES agent architectures on three MoleculeNet [Wu et al., 2018] classification tasks (Table 7).

The architectures achieve mean ROC-AUC of 0.74–0.76 across tasks. The rank correlation between val_bpb and mean ROC-AUC is $\rho = 0.5$ ($n = 3$, $p = 0.67$), consistent with a positive trend but underpowered to reach significance. We additionally evaluate molecular generation: the best agent architecture produces 95.2% valid, 100% unique, and 99.96% novel SMILES strings, confirming that the pretrained representations capture meaningful chemical structure.

5 Practical Implications

The decomposition results translate directly into compute budget allocation guidance.

When to use architecture search vs. HP tuning. Table 8 summarizes a decision framework based on observable domain characteristics. The key predictors are vocabulary size and sequence length: domains with small vocabularies (<100 tokens) and short sequences (<500 tokens) resemble SMILES, where HP tuning alone suffices; domains with large vocabularies (>1K tokens) and long sequences (>1K tokens) resemble NLP, where architecture search contributes the majority of improvement.

Cost-benefit analysis. Each search condition costs approximately \$3.50–5.00 per 100 experiments (5 min $\times$ 100 $\times$ \$0.44/hr GPU $\approx$ \$3.67, plus $\sim$\$1.50 in LLM API costs for agent conditions). Random NAS avoids API costs entirely. For SMILES-like domains, the \$5 HP-only agent achieves the best result (mean best 0.581 bpb); for NLP-like domains, the \$5 full agent achieves the best result (mean best 1.123 bpb). Practitioners working with molecular data can run a full HP search overnight on a single GPU for under \$10.

Table 8: Decision framework: recommended optimization strategy by domain characteristics. Based on the SMILES/protein/NLP decomposition results (Table 4).

Vocab size	Seq length	Resembles	Recommendation
<100	<500	SMILES	HP tuning only
<100	500–1,000	Protein	Either; thin margins
>1,000	>1,000	NLP	Full architecture search

Transferable innovations. Four universally beneficial innovations emerged across all domains and can be applied to any transformer architecture: (1) grouped query attention ($n_{\text{kv_head}} = 1$; 5:1 KV compression with negligible quality loss), (2) gated MLPs (SwiGLU/GeGLU replacing ReluSquared), (3) learned per-layer residual scaling, and (4) value embeddings on every layer rather than alternating layers. These transferred across all domain pairs with <1% degradation.

6 Discussion & Limitations

Why does architecture search hurt on SMILES? We hypothesize that the SMILES domain is "simple enough" (small vocabulary, short sequences, high regularity) that the default architecture is already near-optimal. Architectural modifications waste experimental budget on structural changes that do not improve over the default, while the HP-only agent efficiently tunes learning rates and schedules within a fixed, adequate architecture. This suggests a ceiling effect: when the architecture is not a bottleneck, searching over it is counterproductive.

The clustering-universality paradox. Architectures cluster by domain ($p = 0.004$), yet all innovations transfer freely (41/41 universal). We interpret this as evidence that search dynamics, specifically what the agent explores first conditioned on early training signals, drive apparent specialization. The agent follows different optimization paths for SMILES and NLP data, but the structural innovations it discovers along those paths happen to be universally effective at the ∼10M parameter scale. Whether this universality holds at larger scales, where domain-specific patterns may become more important, remains an open question.

Limitations. (1) *Small model scale* (∼8.6M parameters). At larger scales, domain-specific architectural features (e.g., attention patterns tuned to molecular bonding topology) may become more valuable, potentially changing the decomposition balance. (2) *Short training proxy* (5 minutes). The calibration study yields $\rho = 0.54$, indicating moderate rank correlation with longer training. Architecture rankings could change with extended training budgets, particularly for architectures with different convergence rates. (3) *Single LLM backend* (OpenAI Codex / GPT-5.4). The agent's search behavior, and the resulting decomposition, may differ with other LLMs or with different prompt engineering. (4) *Low statistical power* ($n = 3$–5 runs per condition). The protein track in particular cannot distinguish between conditions. Larger-scale replication would strengthen the protein and NLP conclusions. (5) *SMILES representation only*. SMILES is one of several molecular string representations; SELFIES, InChI, or 3D coordinate representations may yield different decomposition patterns. (6) *Closed-source agent*. The LLM agent operates via API, introducing non-determinism. We mitigate this by running multiple independent replicates per condition, but exact reproduction of individual runs is not possible.

7 Conclusion

We presented the first controlled decomposition of architecture search versus hyperparameter tuning across molecular and language domains, using a 4-condition experimental design with 3,106 experiments. The central finding is that the value of architecture search is domain-dependent: it contributes 81% of improvement for NLP but is counterproductive for SMILES, where HP tuning alone suffices. Despite domain-specific clustering, all 41 discovered innovations transfer universally across domains.

For practitioners: if your domain has a small vocabulary and short sequences, constrain your agent to hyperparameter tuning; if your domain has a large vocabulary and long sequences, invest in architecture search. The framework, experiment logs, and agent prompts are publicly available.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

Angelica Chen, David Dohan, and David So. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. ChemBERTa: Large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

Yann N Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. In *International Conference on Machine Learning*, pages 933–941, 2017.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

Stefan Falkner, Aaron Klein, and Frank Hutter. BOHB: Robust and efficient hyperparameter optimization at scale. In *International Conference on Machine Learning*, pages 1437–1446, 2018.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Yuxuan Hu, Jihao Liu, Ke Wang, Jinliang Zheng, Weikang Shi, Manyuan Zhang, Qi Dou, Rui Liu, Aojun Zhou, and Hongsheng Li. LM-Searcher: Cross-domain neural architecture search with LLMs via unified numerical encoding. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9408–9421, 2025.

John J Irwin and Brian K Shoichet. ZINC—a free database of commercially available compounds for virtual screening. *Journal of Chemical Information and Modeling*, 45(1):177–182, 2005.

Ross Irwin, Spyridon Dimitriadis, Jiazhen He, and Esben Jannik Bjerrum. Chemformer: A pre-trained transformer for computational chemistry. *Machine Learning: Science and Technology*, 3 (1):015022, 2022.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Andrej Karpathy. Autoresearch: Autonomous LLM-driven research. `https://github.com/karpathy/autoresearch`, 2026.

Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. *arXiv preprint arXiv:2206.08896*, 2022.

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Muhammad Umair Nasir, Sam Earle, Julian Togelius, Steven James, and Christopher Cleghorn. LLMatic: Neural architecture search via large language models and quality-diversity optimization. *arXiv preprint arXiv:2306.01102*, 2023.

Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Thomas Wolf, et al. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties. *Nature Machine Intelligence*, 4:1256–1264, 2022.

Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Baris E Suzek, Yuqi Wang, Hongzhan Huang, Peter B McGarvey, Cathy H Wu, and UniProt Consortium. UniRef clusters: a comprehensive and scalable alternative for improving sequence similarity searches. *Bioinformatics*, 31(6):926–932, 2015.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. LLaMA 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

Zhenqin Wu, Bharath Ramsundar, Evan N Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S Pappu, Karl Leswing, and Vijay Pande. MoleculeNet: A benchmark for molecular machine learning. *Chemical Science*, 9(2):513–530, 2018.

Eric Xue, Ke Chen, Zeyi Huang, et al. IMPROVE: Iterative model pipeline refinement and optimization leveraging LLM experts. *arXiv preprint arXiv:2502.18530*, 2025.

Gengmo Zhou, Zhifeng Gao, Qiankun Ding, Hang Zheng, Hongteng Xu, Zhewei Wei, Linfeng Zhang, and Guolin Ke. Uni-Mol: A universal 3D molecular representation learning framework. In *International Conference on Learning Representations*, 2023.

Ethics Statement

This work uses publicly available molecular datasets (ZINC-250K, UniRef50) and a curated web text corpus (FineWeb-Edu). No private, proprietary, or patient-derived data is used. The molecular generation capability demonstrated (95.2% valid SMILES) operates on drug-like molecules from a public database and does not target specific biological pathways or pathogens. The LLM agent operates solely on model architecture code and does not interact with external systems beyond the training loop. The compute footprint is modest: all 3,106 experiments were conducted on a single A10G GPU at a total cost of approximately $200 in GPU time and $50 in API costs. We release all code and experiment logs to support reproducibility.

Reproducibility Statement

Code and data. The complete framework (training script, agent prompts, data preparation pipelines, analysis scripts, and all 3,106 experiment logs including intermediate `train.py` versions and code diffs) is available at `https://github.com/ewijaya/autoresearch-mol`. All datasets are publicly available: ZINC-250K, UniRef50, and FineWeb-Edu.

Experimental setup. All experiments run on a single NVIDIA A10G GPU (24 GB VRAM) with a fixed 5-minute training budget. Random seeds are fixed (`torch.manual_seed(42)`). The random NAS baseline uses deterministic seeds per replicate. Model architecture, optimizer, and training hyperparameters are fully specified in Section 3 and in the released `train.py`.

Statistical analysis. The analysis script (`scripts/analyze_phase2.py`) reads only on-disk results and produces all figures, tables, and hypothesis tests reported in this paper. Bootstrap CIs use 10,000 resamples. Permutation tests use 10,000 permutations. All p-values are reported with both raw and Holm–Bonferroni adjusted values.

Limitations on exact reproducibility. The LLM agent (OpenAI Codex / GPT-5.4) is accessed via API and is non-deterministic: individual agent runs are not exactly reproducible. We address this by running 3–5 independent replicates per condition and reporting aggregate statistics with confidence intervals. The random NAS and fixed default conditions are fully deterministic.

A Supplementary Material

A.1 Hypothesis Summary

Table 9 summarizes the four pre-registered hypotheses, their predictions, statistical tests, and outcomes.

Table 9: Summary of pre-registered hypotheses. All p-values are Holm–Bonferroni adjusted.

Hypothesis	Prediction	Test	Outcome
H1: Arch. clustering	Agent discovers different architectures per domain	Permutation on Gower distance	**Supported** ($p = 0.004$)
H2: Domain knowledge rediscovery	Agent rediscovers known molecular techniques	Code diff vs. 5 known techniques	**Supported**: 4/5 matched
H3: Transfer universality	Domain-specific arch. degrades cross-domain	Transfer + 4 sub-tests	**Not supported**: 41/41 universal ($<1\%$)
H3a: Asymmetric	Degradation is asymmetric	Pairwise degradation	Not supported
H3b: Layer specif.	Late layers domain-specific	Layer freezing	Partially supported
H3c: Length match	Seq. length is artifact	Length truncation	Not supported
H3d: Innovation cls.	35% universal	Binomial test	Contradicted ($p = 2 \times 10^{-19}$)
H4: Arch. vs. HP decomp.	Arch. search adds value beyond HP tuning	Bootstrap CI + Welch's t	**Domain-dep.**: 81% NLP ($p = 0.009$); -51% SMILES ($p = 0.001$)

A.2 Full AUC-OC Values

Table 10 reports AUC-OC and final best val_bpb for every run.

A.3 Decomposition with Bootstrap CIs

Table 11 extends the main-text decomposition (Table 4) with 95% bootstrap confidence intervals computed over 10,000 resamples.

A.4 Additional Figures

Figures 5–11 provide supplementary visualizations referenced in the main text: AUC-OC comparisons, keep rate curves, layer freezing degradation, domain knowledge rediscovery, the permutation null distribution, val_bpb distributions, and training dynamics.

Table 10: Per-run AUC-OC and best val_bpb for all conditions and tracks.

Track	Condition	Run	AUC-OC	Best bpb
SMILES	Agent	1	59.31	0.5918
	Agent	2	58.54	0.5808
	Agent	3	58.80	0.5839
	Agent	4	59.17	0.5892
	Agent	5	59.15	0.5834
	HP-only	1	58.20	0.5807
	HP-only	2	58.14	0.5801
	HP-only	3	58.22	0.5810
	Random NAS	1	59.45	0.5906
	Random NAS	2	59.40	0.5923
	Random NAS	3	59.34	0.5914
	Fixed default	—	59.61	0.5961
Protein	Agent	1	396.86	3.9656
	Agent	2	396.94	3.9684
	Agent	3	396.88	3.9666
	HP-only	1	397.67	3.9901
	HP-only	2	397.07	3.9699
	HP-only	3	396.88	3.9684
	Random NAS	1	397.32	3.9719
	Random NAS	2	397.20	3.9710
	Random NAS	3	397.06	3.9693
	Fixed default	—	397.67	3.9767
NLP	Agent	1	112.78	1.1188
	Agent	2	113.81	1.1277
	Agent	3	112.73	1.1151
	Agent	4	112.97	1.1212
	Agent	5	113.71	1.1314
	HP-only	1	114.75	1.1462
	HP-only	2	114.90	1.1470
	HP-only	3	114.89	1.1477
	Random NAS	1	113.18	1.1297
	Random NAS	2	113.32	1.1301
	Random NAS	3	113.48	1.1306
	Fixed default	—	115.28	1.1528

Table 11: Full decomposition with 95% bootstrap confidence intervals.

Track	HP contrib.	HP 95% CI	Arch contrib.	Arch 95% CI
SMILES	0.0155 (151%)	[0.0151, 0.0159]	−0.0052 (−51%)	[−0.0089, −0.0018]
Protein	0.0005 (6%)	[−0.0134, 0.0083]	0.0093 (94%)	[0.0011, 0.0229]
NLP	0.0058 (19%)	[0.0050, 0.0066]	0.0241 (81%)	[0.0188, 0.0292]

A.5 Agent Prompts

The full agent system prompt (`program.md`) and HP-only prompt (`program_hponly.md`) are included in the released code repository. The key difference: the HP-only prompt adds the constraint "Do NOT change model architecture: no new layers, no attention pattern changes, no activation function changes, no model structure changes."

A.6 All Adjusted p-values

Table 12 reports all 30 frequentist tests with raw and Holm–Bonferroni adjusted p-values, grouped by family.

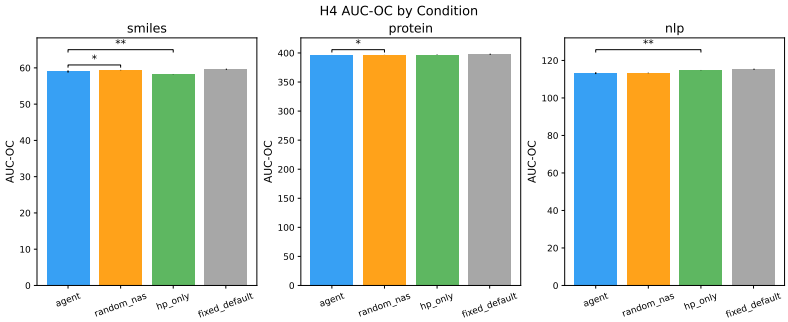


Figure 5: AUC-OC comparison across conditions and tracks.

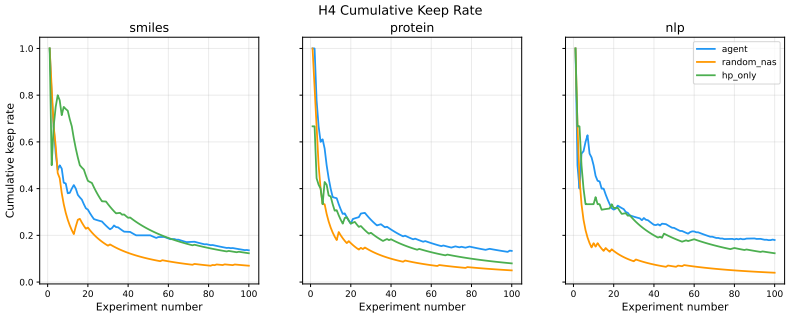


Figure 6: Cumulative keep rate curves by condition.

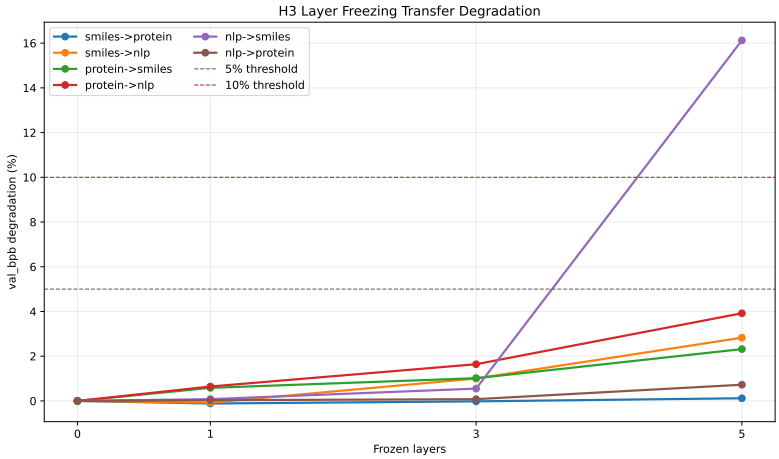


Figure 7: Layer freezing degradation curves. Degradation increases monotonically with the number of frozen layers. Early layers (low freeze level) transfer cleanly; late layers show mild domain specificity.

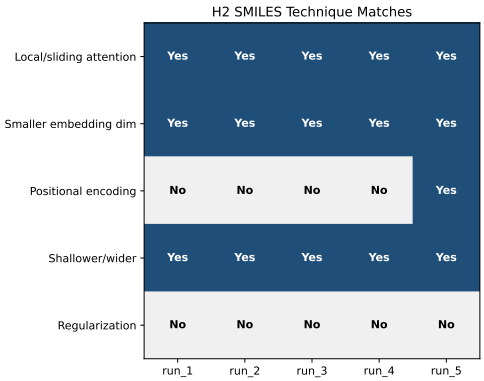


Figure 8: Domain knowledge rediscovery: technique $\times$ run binary matrix for SMILES agent runs. 4 of 5 known molecular techniques are independently discovered.

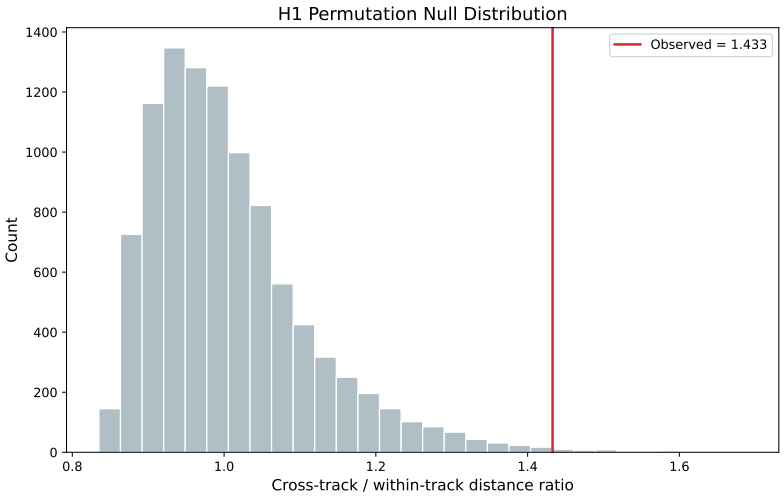
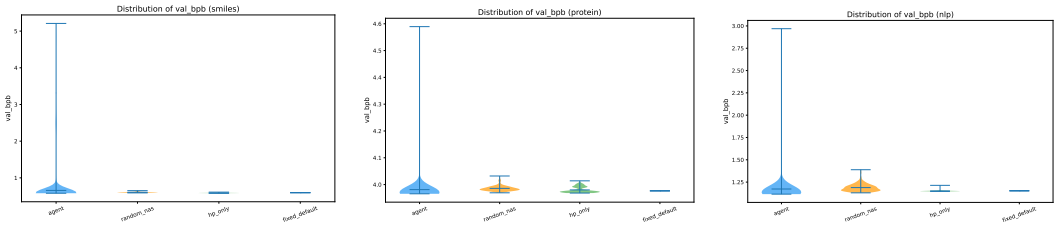
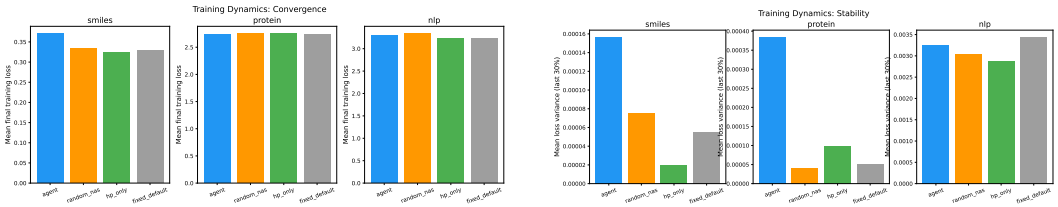


Figure 9: Permutation null distribution for the cross-track/within-track distance ratio. The observed ratio (1.43) falls in the extreme tail ($p = 0.004$).



(a) SMILES (b) Protein (c) NLP

Figure 10: Distribution of val_bpb by condition and track (violin plots).



(a) Convergence rate

(b) Training stability

Figure 11: Training dynamics across conditions. Agent and baseline conditions show comparable convergence rates and training stability, confirming that performance differences arise from architectural and hyperparameter choices, not training dynamics artifacts.

Table 12: Complete multiple comparison correction. Holm–Bonferroni adjusted p-values within logical families.

Family	**Test**	**Raw** p	**Adj.** p
H1	Permutation test	0.0037	0.0037
H4 decomp.	SMILES HP contrib.	0.0003	0.0012
	NLP arch contrib.	0.0017	0.0085
	NLP HP contrib.	0.0054	0.0216
H4 SMILES	Agent vs. fixed default	0.0001	0.0006
	HP-only vs. fixed default	0.0001	0.0006
	Agent vs. HP-only	0.0036	0.0145
	Agent vs. NAS	0.0146	0.0438
H4 NLP	NAS vs. fixed default	0.0001	0.0003
	Agent vs. fixed default	0.0002	0.0011
	Agent vs. HP-only	0.0013	0.0051
	HP-only vs. fixed default	0.0015	0.0060